

Verified Random vs AI-Guided Fuzzing Findings

Separate results from 108,000 random inputs and four 36,000-input AI experiments

Interim analysis - no cross-model AI average used

Strict verification statement

Every result in this document was recomputed from the uploaded raw JSONL and metadata archives. The analysis parsed 252,000 target-execution records, verified ZIP integrity, checked group sizes, and found zero invalid JSONL records. Results are intentionally kept separate by random batch and AI model.

Prepared August 4, 2026

Research use: authorized software-testing experiments only

1. Scope and Primary Research Question

The primary question is not which LLM is best. The central question is whether AI-guided fuzzing changes the balance between valid-input generation, error-path exploration, source-line coverage, input diversity, and operational cost when compared with random fuzzing under matched 500-input target/trial portfolios.

Fair comparison rule

For every target and trial, the five random strategies are combined as $5 \times 100 = 500$ random inputs. That 500-input random portfolio is compared with one 500-input AI model group. The combined 108,000 random records are not treated as one three-times-larger opponent to a 36,000-input AI model.

2. Data Integrity and Experiment Structure

Dataset	Records	JSONL	Metadata	Groups	Inputs/group	SHA-256 prefix
Random trials 1-9	108,000	1080	0	1080	100	DE0C59BAF4634041...
Gemma 4 31B	36,000	72	72	72	500	6D8008338D4B513A...
GPT-OSS 120B	36,000	72	72	72	500	6540D787FD9DD327...
Qwen 3.5 122B	36,000	72	72	72	500	1E6709C1959D0C36...
Qwen 3.6 35B	36,000	72	72	72	500	BBC0EFE9DE7AC463...

Complete SHA-256 hashes are listed in Appendix D. Random data contains 24 targets, five strategies, and nine trials. Each AI archive contains 24 targets, three trials, and 500 executed inputs per target/trial.

3. Strict Measurement Definitions

- Valid input: a record with success=true, meaning the target returned without raising an exception.
- Error input: a record with success=false, meaning the target raised an exception or the random runner recorded a safety rejection.
- A valid input is not automatically semantically correct. An error input is not automatically evidence of a software defect.
- Target-execution errors are separated from LLM generation and infrastructure failures.
- Fair coverage is the union of covered source lines within each 500-input target/trial portfolio, divided by the corrected target-level source-line denominator.

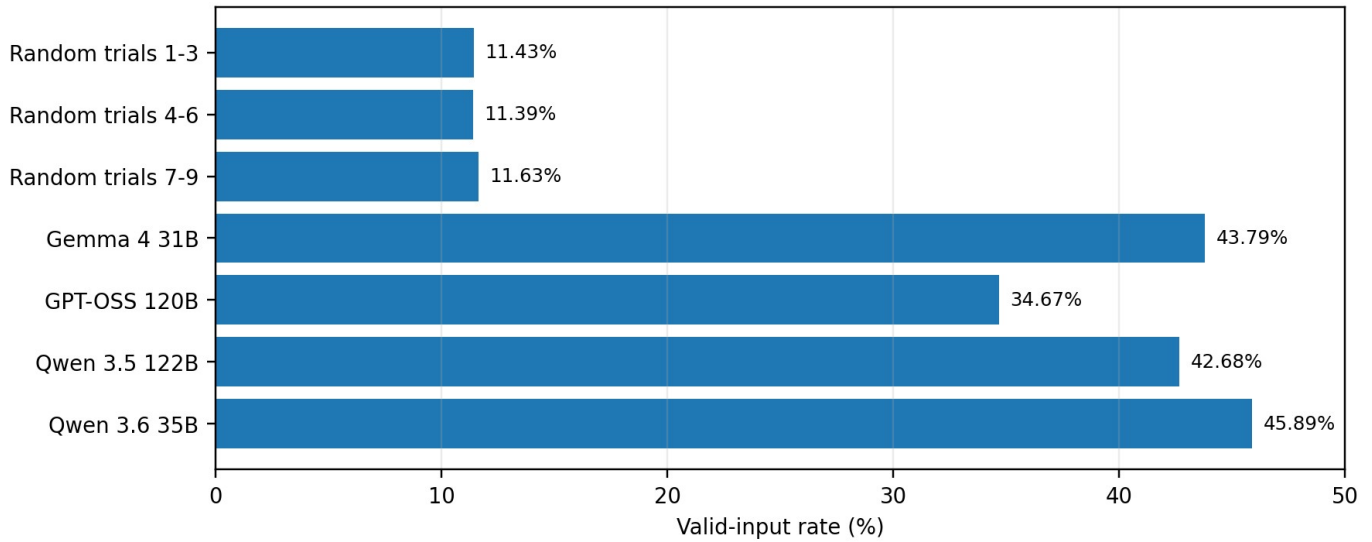
Coverage denominator correction

The raw Week 8 records contain an earlier per-function source_line_count value that is not suitable for the final comparison. Coverage in this report uses the corrected target-level denominators from week8_random_summary(1).json. This prevents coverage values above 100% and applies the same denominator to random and AI records for each target.

4. Main Result Overview - Separate Results

Method	Inputs	Valid	Valid rate	Errors	Error rate	Fair avg coverage
Random trials 1-3	36,000	4,113	11.425%	31,887	88.575%	73.860%
Random trials 4-6	36,000	4,102	11.394%	31,898	88.606%	73.666%
Random trials 7-9	36,000	4,188	11.633%	31,812	88.367%	73.996%

Method	Inputs	Valid	Valid rate	Errors	Error rate	Fair avg coverage
Gemma 4 31B	36,000	15,763	43.786%	20,237	56.214%	73.359%
GPT-OSS 120B	36,000	12,483	34.675%	23,517	65.325%	73.396%
Qwen 3.5 122B	36,000	15,364	42.678%	20,636	57.322%	74.144%
Qwen 3.6 35B	36,000	16,519	45.886%	19,481	54.114%	74.428%



Immediate interpretation

- All four AI models produced substantially higher valid-input rates than every random batch.
- The three random batches were highly stable: valid rates ranged from 11.394% to 11.633%.
- Coverage stayed close across all methods: 73.359% to 74.428%.
- Therefore, the strongest current AI result is improved valid-input generation, not a large universal coverage gain.

5. Random Baseline - Three Independent Batches

Batch	Inputs	Valid	Valid rate	Errors	Fair coverage	Exact error signatures
trials 1-3	36,000	4,113	11.425%	31,887	73.860%	235
trials 4-6	36,000	4,102	11.394%	31,898	73.666%	213
trials 7-9	36,000	4,188	11.633%	31,812	73.996%	213

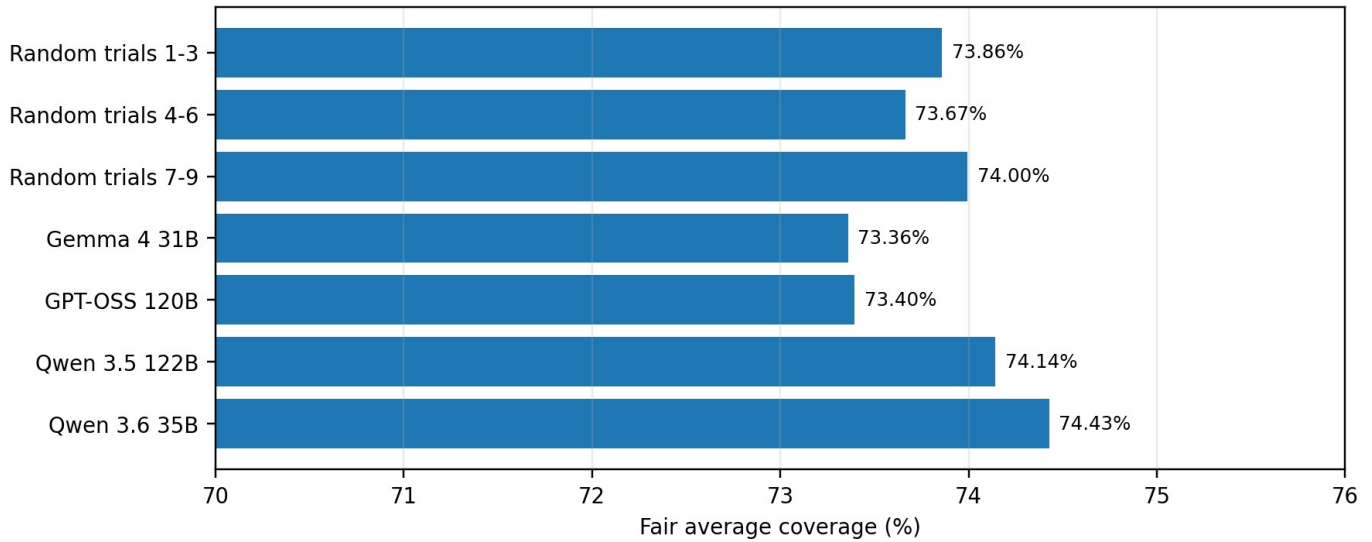
Nine-trial stability summary
 Across all nine random trials: 12,403 valid inputs out of 108,000 (11.484%). The three 36,000-input batches differ by only 0.239 percentage points in valid rate and 0.329 points in fair coverage. The original trials 1-3 result was therefore not an unusually lucky or unlucky random sample.

Per-trial random stability

Trial	Inputs	Valid	Valid rate	Fair avg coverage
1	12,000	1,351	11.258%	73.883%
2	12,000	1,385	11.542%	73.968%
3	12,000	1,377	11.475%	73.729%
4	12,000	1,413	11.775%	73.605%
5	12,000	1,321	11.008%	73.696%
6	12,000	1,368	11.400%	73.697%
7	12,000	1,399	11.658%	74.014%
8	12,000	1,410	11.750%	74.098%
9	12,000	1,379	11.492%	73.875%

6. AI-Guided Results - Each Model Kept Separate

Model	Valid	Valid rate	Errors	Error rate	Fair coverage	Cross-trial unique
Gemma 4 31B	15,763	43.786%	20,237	56.214%	73.359%	81.950%
GPT-OSS 120B	12,483	34.675%	23,517	65.325%	73.396%	86.839%
Qwen 3.5 122B	15,364	42.678%	20,636	57.322%	74.144%	88.333%
Qwen 3.6 35B	16,519	45.886%	19,481	54.114%	74.428%	84.281%



Per-model trial variation

Model	Trial 1 / 2 / 3 valid rates	Valid-rate range	Trial 1 / 2 / 3 coverage
Gemma 4 31B	43.925% / 44.083% / 43.350%	43.350% - 44.083%	73.387% / 73.375% / 73.315%
GPT-OSS 120B	34.625% / 35.217% / 34.183%	34.183% - 35.217%	72.693% / 74.863% / 72.632%
Qwen 3.5 122B	41.658% / 42.775% / 43.600%	41.658% - 43.600%	74.162% / 73.967% / 74.303%
Qwen 3.6 35B	45.800% / 46.117% / 45.742%	45.742% - 46.117%	74.296% / 74.532% / 74.454%

Careful interpretation

AI valid rates ranged from 34.675% to 45.886%, approximately 2.98x to 4.03x the nine-trial random valid rate. Coverage differences were much smaller. Qwen 3.5 and Qwen 3.6 were slightly above the random batches; Gemma and GPT-OSS were slightly below them. These differences should not be described as a broad coverage victory.

7. Error-Type Analysis

Error rate is important, but a higher error rate is not automatically better. Repeated ValueError rejections, specialized runtime exceptions, and infrastructure failures must be distinguished.

Method	ValueError	SyntaxError	KeyError	ZeroDivisionError	IndentationError	TypeError	SafetyError
Random trials 1-3	30,578	1,107	111	5	57	0	29
Random trials 4-6	30,567	1,137	120	10	36	1	27
Random trials 7-9	30,460	1,160	100	6	54	1	31
Gemma 4 31B	18,983	580	592	25	57	0	0
GPT-OSS 120B	22,217	798	294	176	29	3	0
Qwen 3.5 122B	19,519	476	565	42	34	0	0
Qwen 3.6 35B	17,972	412	911	129	57	0	0

Error composition findings

- Random trials 1-3: ValueError represented 95.89% of all target-execution errors.
- Random trials 4-6: ValueError represented 95.83% of all target-execution errors.
- Random trials 7-9: ValueError represented 95.75% of all target-execution errors.
- Gemma 4 31B: ValueError represented 93.80% of all target-execution errors.
- GPT-OSS 120B: ValueError represented 94.47% of all target-execution errors.
- Qwen 3.5 122B: ValueError represented 94.59% of all target-execution errors.
- Qwen 3.6 35B: ValueError represented 92.25% of all target-execution errors.

What the counts support

Random fuzzing generated more SyntaxError outcomes, especially in calculator and arithmetic inputs. AI models generated proportionally more KeyError and ZeroDivisionError outcomes. This demonstrates a change in error mix, but not automatic defect discovery. Many errors are expected validation or rejection behavior.

Three failure categories that must remain separate

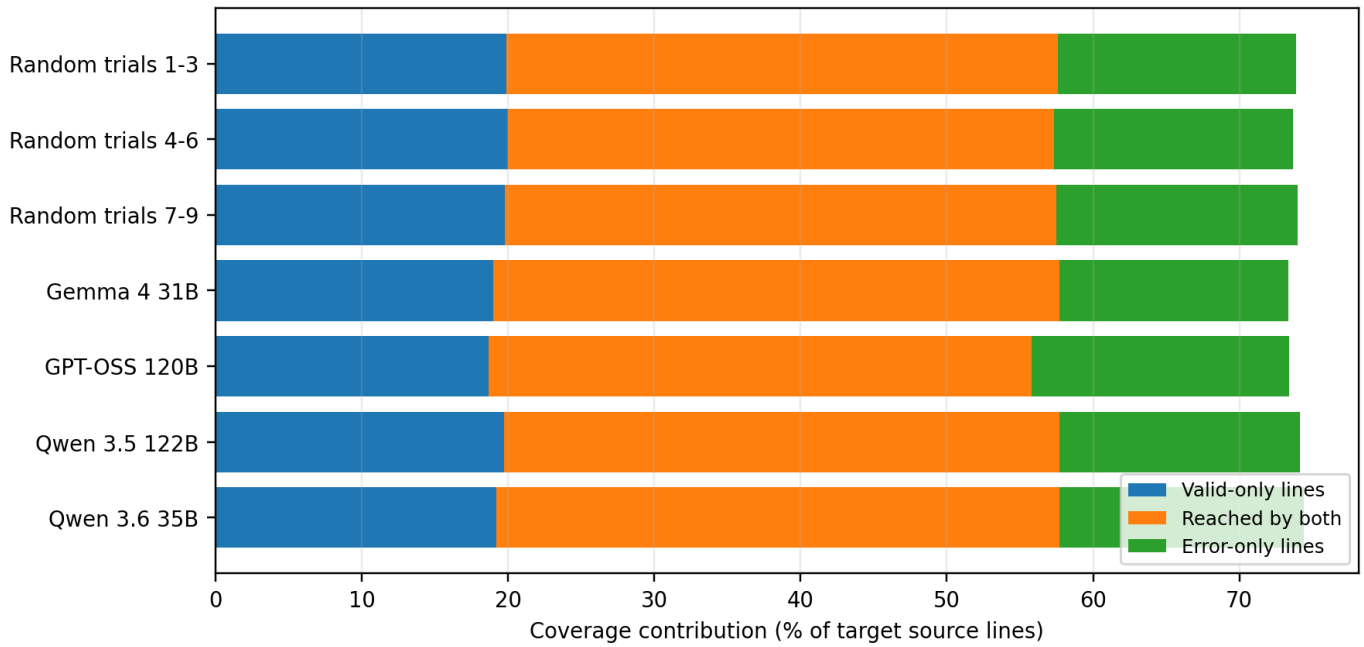
- Expected input rejection: common ValueError outcomes caused by invalid syntax, format, or constraints.
- Target runtime exception: KeyError, ZeroDivisionError, TypeError, SyntaxError, and related target behavior.
- AI pipeline or infrastructure failure: malformed LLM JSON, HTTP 502, DNS failure, connection reset, or no usable response. These are not target-program errors.

8. Why Error-Producing Inputs Still Matter

Coverage was decomposed into lines reached only by valid inputs, only by error-producing inputs, and by both. The three components sum to the fair total coverage for each 500-input group.

Method	Total	Valid-only	Error-only	Both
Random trials 1-3	73.860%	19.862%	16.268%	37.731%
Random trials 4-6	73.666%	19.995%	16.362%	37.309%
Random trials 7-9	73.996%	19.801%	16.533%	37.661%
Gemma 4 31B	73.359%	18.973%	15.678%	38.708%
GPT-OSS 120B	73.396%	18.652%	17.588%	37.156%
Qwen 3.5 122B	74.144%	19.741%	16.463%	37.941%

Method	Total	Valid-only	Error-only	Both
Qwen 3.6 35B	74.428%	19.226%	16.746%	38.456%



Central finding for the paper

Error-producing inputs uniquely reached approximately 15.7% to 17.6% of target source lines. Therefore, evaluating fuzzing only by valid-input rate would omit a meaningful part of the exercised program behavior.

9. Unique Error Signatures and Normalization

Dataset	Exact target/type/message signatures	Normalized signatures
Random trials 1-3	235	Not used separately
Random trials 4-6	213	Not used separately
Random trials 7-9	213	Not used separately
Gemma 4 31B	162	141
GPT-OSS 120B	195	150
Qwen 3.5 122B	185	142
Qwen 3.6 35B	308	144
Random, all trials 1-9	341	176

The normalization is deliberately conservative. It replaces only unknown template-variable names and the line number inside the Python "<unknown>, line N" suffix. No broader semantic clustering was applied.

Why exact counts can mislead

Qwen 3.6 produced 308 exact signatures, but many were `template_renderer` `KeyErrors` that differed only by the unknown variable name. After conservative normalization, the four AI models had 141-150 signatures and the full nine-trial random set had 176. The data does not support claiming that Qwen 3.6 discovered twice as many fundamentally different failures.

10. Difficulty-Level Valid Rates

Method	Easy	Moderate	Hard
Random trials 1-3	10.692%	15.550%	8.033%
Random trials 4-6	10.692%	15.692%	7.800%
Random trials 7-9	10.967%	15.992%	7.942%
Gemma 4 31B	30.958%	63.892%	36.508%
GPT-OSS 120B	25.367%	58.533%	20.125%
Qwen 3.5 122B	31.942%	61.575%	34.517%
Qwen 3.6 35B	34.675%	61.967%	41.017%

AI improvements were largest on moderate and hard structured parsers. GPT-OSS remained higher than random but was weaker than the other AI models on hard targets.

11. Target-Level Transparency

- Gemma 4 31B was higher on 23 of 24 targets and lower on 1: `template_renderer`: random 41.33% vs AI 37.47%.
- GPT-OSS 120B was higher on 21 of 24 targets and lower on 3: `http_request_parser`: random 3.36% vs AI 0.93%; `phone_validator`: random 1.13% vs AI 1.00%; `template_renderer`: random 41.33% vs AI 9.27%.
- Qwen 3.5 122B was higher on 23 of 24 targets and lower on 1: `template_renderer`: random 41.33% vs AI 39.33%.
- Qwen 3.6 35B was higher on 23 of 24 targets and lower on 1: `template_renderer`: random 41.33% vs AI 25.07%.

It would be inaccurate to claim that AI outperformed random fuzzing on every target. Full target-level valid-rate and coverage tables are included in Appendices A and B.

12. Input Diversity

Method	Unique within each 500-input group	Cross-trial uniqueness
Random trials 1-3	93.117%	88.911%
Random trials 4-6	92.931%	88.683%
Random trials 7-9	92.897%	88.611%
Gemma 4 31B	100.000%	81.950%
GPT-OSS 120B	100.000%	86.839%
Qwen 3.5 122B	100.000%	88.333%
Qwen 3.6 35B	100.000%	84.281%

The AI fuzzer enforces uniqueness within each 500-input group, so the 100% within-group result is partly an implementation rule. Across repeated trials, random fuzzing was more diverse than Gemma, GPT-OSS, and Qwen 3.6, and approximately tied with Qwen 3.5.

13. Execution Time and AI Operational Limitations

Method	Mean target exec/input	Total target exec	Recorded generation	Generation-round failures	Pre-exec rejected
Random trials 1-3	4.790e-05 s	1.724 s	-	-	-
Random trials 4-6	4.718e-05 s	1.699 s	-	-	-
Random trials 7-9	4.827e-05 s	1.738 s	-	-	-
Gemma 4 31B	9.053e-05 s	3.259 s	2.907 h	528	46
GPT-OSS 120B	9.659e-05 s	3.477 s	6.527 h	205	59
Qwen 3.5 122B	9.673e-05 s	3.482 s	3.724 h	840	51
Qwen 3.6 35B	1.142e-04 s	4.112 s	1.516 h	343	23

Target execution remained very fast; AI generation dominated experiment time. Recorded generation time covers the final successful metadata groups and does not fully include earlier failed full-group attempts that were later rerun.

AI generation-failure categories

- Gemma 4 31B - Malformed or unusable JSON: 397, DNS error: 113, No new unique inputs: 17, Connection error: 1.
- GPT-OSS 120B - Malformed or unusable JSON: 131, No usable response content: 44, No new unique inputs: 30.
- Qwen 3.5 122B - Malformed or unusable JSON: 791, No new unique inputs: 49.
- Qwen 3.6 35B - Malformed or unusable JSON: 184, HTTP/backend error: 97, No new unique inputs: 62.

14. Strict Conclusions Supported by the Data

- AI substantially improves valid-input generation. This finding is stable against all three independent random batches.
- Overall fair coverage is nearly the same across methods. The AI coverage advantage is small and model-dependent.
- Error-producing inputs remain essential because they uniquely cover approximately 16-18% of target source lines.
- Higher error count is not automatically better; most errors are repeated validation failures and many exact messages share one behavioral pattern.
- AI changes the error mix, producing proportionally more KeyError and arithmetic runtime failures while random produces more SyntaxError outcomes.
- AI has meaningful operational costs: remote generation time, malformed outputs, retries, and infrastructure dependence.
- The random baseline is stable across nine trials, confirming the original three-trial result.

15. Limitations and Required Next Step

- The archives do not embed a Git commit SHA or source-file hashes for all 24 targets. Line numbering is consistent, but source-version identity cannot be cryptographically proven from the archives alone.
- success=true measures exception-free execution, not full semantic correctness of the returned result.
- Exception type and normalized message signature are useful but incomplete proxies for distinct defects. Stack traces, fault locations, and manually defined expected-rejection categories would strengthen defect classification.
- Random has nine trials while each AI model has three. This is transparent robustness replication, not symmetric replication.
- No cross-model AI average is presented in this interim document. Separate model findings should remain visible before any later aggregate is added to the paper.

Next reproducibility step

Preserve this analysis as a repository script and machine-readable tables. Future manifests should record the Git commit SHA, target source-file SHA-256 hashes, runner version, prompt version, random seed, and exact model identifier.

Appendix A. Target-Level Valid-Input Rates

Target	Diff.	Random 1-9	Gemma	GPT-OSS	Qwen 3.5	Qwen 3.6
arithmetic_script_parser	hard	1.09%	22.53%	23.80%	31.00%	31.53%
boolean_expression_parser	hard	2.04%	49.53%	40.13%	43.73%	56.13%
calculator	easy	10.07%	51.13%	41.40%	70.07%	59.13%
color_code_validator	easy	2.11%	24.40%	14.07%	20.20%	28.67%
command_parser	hard	4.60%	21.60%	9.47%	13.00%	24.93%
config_parser	moderate	19.07%	70.33%	35.13%	56.00%	50.80%
csv_parser	moderate	27.24%	70.93%	78.93%	69.80%	69.07%
date_parser	easy	1.49%	13.67%	4.80%	13.07%	22.80%
email_validator	easy	8.36%	50.27%	33.87%	43.53%	48.00%
file_path_parser	moderate	33.27%	69.93%	70.07%	62.20%	72.73%
http_request_parser	hard	3.36%	54.40%	0.93%	64.47%	69.40%
json_parser	moderate	2.96%	49.40%	42.47%	48.07%	48.93%
log_line_parser	moderate	8.24%	54.73%	56.27%	64.47%	61.13%
markdown_link_parser	moderate	4.00%	41.40%	30.60%	50.93%	45.20%
mini_protocol_parser	hard	3.82%	27.53%	24.53%	6.27%	28.40%
password_checker	easy	39.20%	51.07%	74.33%	55.40%	53.53%
phone_validator	easy	1.13%	1.60%	1.00%	4.60%	7.07%
query_string_parser	moderate	24.62%	64.60%	67.93%	53.20%	60.60%
sql_where_parser	hard	4.64%	23.60%	13.87%	41.00%	49.53%
template_renderer	hard	41.33%	37.47%	9.27%	39.33%	25.07%
url_parser	moderate	6.56%	89.80%	86.87%	87.93%	87.27%
username_validator	easy	21.44%	40.73%	26.60%	36.40%	39.27%
xml_like_parser	hard	2.51%	55.40%	39.00%	37.33%	43.13%
zip_code_validator	easy	2.47%	14.80%	6.87%	12.27%	18.93%

Appendix B. Fair 500-Input Target Coverage

Target	Lines	Random 1-9	Gemma	GPT-OSS	Qwen 3.5	Qwen 3.6
arithmetic_script_parser	59	77.97%	77.97%	77.97%	77.97%	77.97%
boolean_expression_parser	44	86.36%	86.36%	86.36%	86.36%	86.36%
calculator	32	37.50%	37.50%	37.50%	37.50%	37.50%
color_code_validator	18	72.22%	66.67%	72.22%	72.22%	72.22%
command_parser	40	72.50%	72.50%	72.50%	70.83%	69.17%
config_parser	32	85.76%	87.50%	87.50%	87.50%	87.50%
csv_parser	24	66.67%	75.00%	72.22%	72.22%	75.00%
date_parser	24	62.50%	62.50%	62.50%	62.50%	62.50%
email_validator	23	74.88%	69.57%	78.26%	73.91%	73.91%
file_path_parser	26	80.77%	84.62%	82.05%	80.77%	80.77%
http_request_parser	43	86.30%	86.82%	54.26%	85.27%	84.50%
json_parser	19	78.36%	78.95%	77.19%	78.95%	78.95%
log_line_parser	22	68.18%	72.73%	72.73%	69.70%	71.21%
markdown_link_parser	21	71.43%	76.19%	74.60%	74.60%	76.19%
mini_protocol_parser	46	85.99%	86.96%	86.96%	84.06%	86.96%
password_checker	24	79.17%	77.78%	79.17%	79.17%	79.17%
phone_validator	22	59.09%	54.55%	59.09%	59.09%	59.09%
query_string_parser	28	85.71%	82.14%	85.71%	85.71%	85.71%
sql_where_parser	47	74.70%	76.60%	76.60%	76.60%	76.60%
template_renderer	32	65.62%	62.50%	65.62%	65.62%	65.62%
url_parser	20	80.00%	76.67%	80.00%	80.00%	80.00%
username_validator	21	76.19%	71.43%	76.19%	74.60%	76.19%
xml_like_parser	30	80.00%	80.00%	80.00%	80.00%	78.89%
zip_code_validator	14	64.29%	57.14%	64.29%	64.29%	64.29%

Random coverage averages nine 500-input target/trial portfolios. Each AI value averages three 500-input target/trial groups. The Lines column is the corrected target-level source-line denominator used for every method.

Appendix C. Trial-Level Results

Method	Trial	Inputs	Valid	Valid rate	Fair avg coverage
Random	1	12,000	1,351	11.258%	73.883%
Random	2	12,000	1,385	11.542%	73.968%
Random	3	12,000	1,377	11.475%	73.729%
Random	4	12,000	1,413	11.775%	73.605%
Random	5	12,000	1,321	11.008%	73.696%
Random	6	12,000	1,368	11.400%	73.697%
Random	7	12,000	1,399	11.658%	74.014%
Random	8	12,000	1,410	11.750%	74.098%
Random	9	12,000	1,379	11.492%	73.875%
Gemma 4 31B	1	12,000	5,271	43.925%	73.387%
Gemma 4 31B	2	12,000	5,290	44.083%	73.375%
Gemma 4 31B	3	12,000	5,202	43.350%	73.315%
GPT-OSS 120B	1	12,000	4,155	34.625%	72.693%
GPT-OSS 120B	2	12,000	4,226	35.217%	74.863%
GPT-OSS 120B	3	12,000	4,102	34.183%	72.632%
Qwen 3.5 122B	1	12,000	4,999	41.658%	74.162%
Qwen 3.5 122B	2	12,000	5,133	42.775%	73.967%
Qwen 3.5 122B	3	12,000	5,232	43.600%	74.303%
Qwen 3.6 35B	1	12,000	5,496	45.800%	74.296%
Qwen 3.6 35B	2	12,000	5,534	46.117%	74.532%
Qwen 3.6 35B	3	12,000	5,489	45.742%	74.454%

Appendix D. Source Archives and Complete SHA-256 Hashes

Dataset	Filename	SHA-256
Random trials 1-9	week8_random_all_trials1_9_108000_results.zip	DE0C59BAF4634041F8D4A85DDA16D3A5C28423EB49BE63ECD596D3CDF48BC9CE

Dataset	Filename	SHA-256
Gemma 4 31B	google_gemma_4_31b_36000_results.zip	6D8008338D4B513ADBD25BCABBA30C9D92F04C519D4DD808B4BDB803DF303B64
GPT-OSS 120B	openai_gpt_oss_120b_36000_results.zip	6540D787FD9DD327FEDD86C4BAC4FF7EB65B6CA6B00B15C9EC071BB50F6004E1
Qwen 3.5 122B	qwen_qwen3_5_122b_36000_results.zip	1E6709C1959D0C36EBA8F59008DF8B5A05BF362A4D55B6138DB32F85B311892A
Qwen 3.6 35B	qwen_qwen3_6_35b_36000_results.zip	BBC0EFE9DE7AC4639443D367C343EBAF05BDD119B6AC6AED37D10D4FA1891665
Coverage denominator source	week8_random_summary(1).json	8383D4FAD9459CFFD5FD416843EBD7118A34F05D714618A6B268A9F6A493F076